

```
# =====
# NEURAL STYLE TRANSFER PROJECT
# Google Colab
# =====

# Install dependencies
!pip install tensorflow matplotlib pillow -q

import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from google.colab import files
import time

# =====
# IMAGE LOADING FUNCTION
# =====

def load_img(path_to_img):

    max_dim = 512

    img = Image.open(path_to_img)

    long = max(img.size)

    scale = max_dim / long

    img = img.resize(
        (round(img.size[0] * scale),
         round(img.size[1] * scale))
    )

    img = np.array(img)

    img = tf.convert_to_tensor(img)

    img = tf.image.convert_image_dtype(img, tf.float32)

    img = img[tf.newaxis, :]

    return img
```

```

# =====
# DISPLAY FUNCTION
# =====

def imshow(image, title=None):

    if len(image.shape) > 3:
        image = tf.squeeze(image, axis=0)

    plt.imshow(image)

    if title:
        plt.title(title)

# =====
# UPLOAD CONTENT IMAGE
# =====

print("Upload CONTENT image")

uploaded = files.upload()

content_path = list(uploaded.keys())[0]

# =====
# UPLOAD STYLE IMAGE
# =====

print("\nUpload STYLE image")

uploaded = files.upload()

style_path = list(uploaded.keys())[0]

# =====
# LOAD IMAGES
# =====

content_image = load_img(content_path)

style_image = load_img(style_path)

# =====
# LOAD VGG19 MODEL

```

```

# =====

vgg = tf.keras.applications.VGG19(
    include_top=False,
    weights='imagenet'
)

vgg.trainable = False

# =====
# STYLE TRANSFER MODEL
# =====

content_layers = ['block5_conv2']

style_layers = [
    'block1_conv1',
    'block2_conv1',
    'block3_conv1',
    'block4_conv1',
    'block5_conv1'
]

num_content_layers = len(content_layers)
num_style_layers = len(style_layers)

# =====
# MODEL FUNCTION
# =====

def vgg_layers(layer_names):

    outputs = [vgg.get_layer(name).output for name in layer_names]

    model = tf.keras.Model([vgg.input], outputs)

    return model

# =====
# GRAM MATRIX
# =====

def gram_matrix(input_tensor):

    result = tf.linalg.einsum(

```

```

        'bijk,bijd->bcd',
        input_tensor,
        input_tensor
    )

    input_shape = tf.shape(input_tensor)

    num_locations = tf.cast(
        input_shape[1] * input_shape[2],
        tf.float32
    )

    return result / num_locations

# =====
# STYLE CONTENT MODEL
# =====

class StyleContentModel(tf.keras.models.Model):

    def __init__(self, style_layers, content_layers):

        super(StyleContentModel, self).__init__()

        self.vgg = vgg_layers(style_layers + content_layer

        self.style_layers = style_layers

        self.content_layers = content_layers

        self.num_style_layers = len(style_layers)

        self.vgg.trainable = False

    def call(self, inputs):

        inputs = inputs * 255.0

        preprocessed_input = tf.keras.applications.vgg19.p

        outputs = self.vgg(preprocessed_input)

        style_outputs, content_outputs = (
            outputs[:self.num_style_layers],
            outputs[self.num_style_layers:]

```

```

    )

    style_outputs = [
        gram_matrix(style_output)
        for style_output in style_outputs
    ]

    content_dict = {
        content_name: value
        for content_name, value
        in zip(self.content_layers, content_outputs)
    }

    style_dict = {
        style_name: value
        for style_name, value
        in zip(self.style_layers, style_outputs)
    }

    return {
        'content': content_dict,
        'style': style_dict
    }

# =====
# EXTRACT STYLE + CONTENT
# =====

extractor = StyleContentModel(style_layers, content_layers)

style_targets = extractor(style_image)['style']

content_targets = extractor(content_image)['content']

# =====
# GENERATED IMAGE
# =====

image = tf.Variable(content_image)

# =====
# OPTIMIZER
# =====

opt = tf.optimizers.Adam(

```

```

        learning_rate=0.02
    )

    # =====
    # LOSS FUNCTION
    # =====

    style_weight = 1e-2
    content_weight = 1e4

    def style_content_loss(outputs):

        style_outputs = outputs['style']
        content_outputs = outputs['content']

        style_loss = tf.add_n([
            tf.reduce_mean(
                (style_outputs[name] - style_targets[name])**2
            )
            for name in style_outputs.keys()
        ])

        style_loss *= style_weight / num_style_layers

        content_loss = tf.add_n([
            tf.reduce_mean(
                (content_outputs[name] - content_targets[name])
            )
            for name in content_outputs.keys()
        ])

        content_loss *= content_weight / num_content_layers

        loss = style_loss + content_loss

        return loss

    # =====
    # TRAIN STEP
    # =====

    @tf.function()
    def train_step(image):

        with tf.GradientTape() as tape:

```

```
        outputs = extractor(image)

        loss = style_content_loss(outputs)

        grad = tape.gradient(loss, image)

        opt.apply_gradients([(grad, image)])

        image.assign(tf.clip_by_value(image, 0.0, 1.0))

# =====
# TRAINING
# =====

epochs = 5
steps_per_epoch = 20

step = 0

for n in range(epochs):

    for m in range(steps_per_epoch):

        step += 1

        train_step(image)

        print(".", end='')

    print(f"\nTrain step: {step}")

# =====
# SHOW RESULT
# =====

plt.figure(figsize=(10,10))

imshow(image, 'Styled Image')

plt.axis('off')

plt.show()

# =====
```

```
# SAVE OUTPUT
# =====

output = tf.squeeze(image) * 255

output = np.array(output, dtype=np.uint8)

output_image = Image.fromarray(output)

output_image.save("styled_output.jpg")

files.download("styled_output.jpg")

print("\nStyled image downloaded successfully!")
```



Upload CONTENT image

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving IMG\_20230601\_212456\_671.jpg to IMG\_20230601\_212456\_671.jpg

Upload STYLE image

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving star.jfif to star.jfif

.....

Train step: 20

.....

Train step: 40

.....

Train step: 60

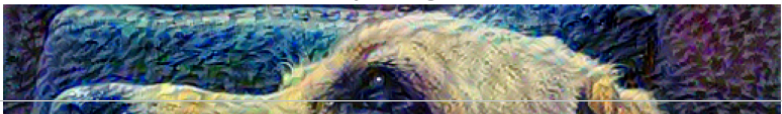
.....

Train step: 80

.....

Train step: 100

Styled Image



[Start coding](#) or [generate](#) with AI.

